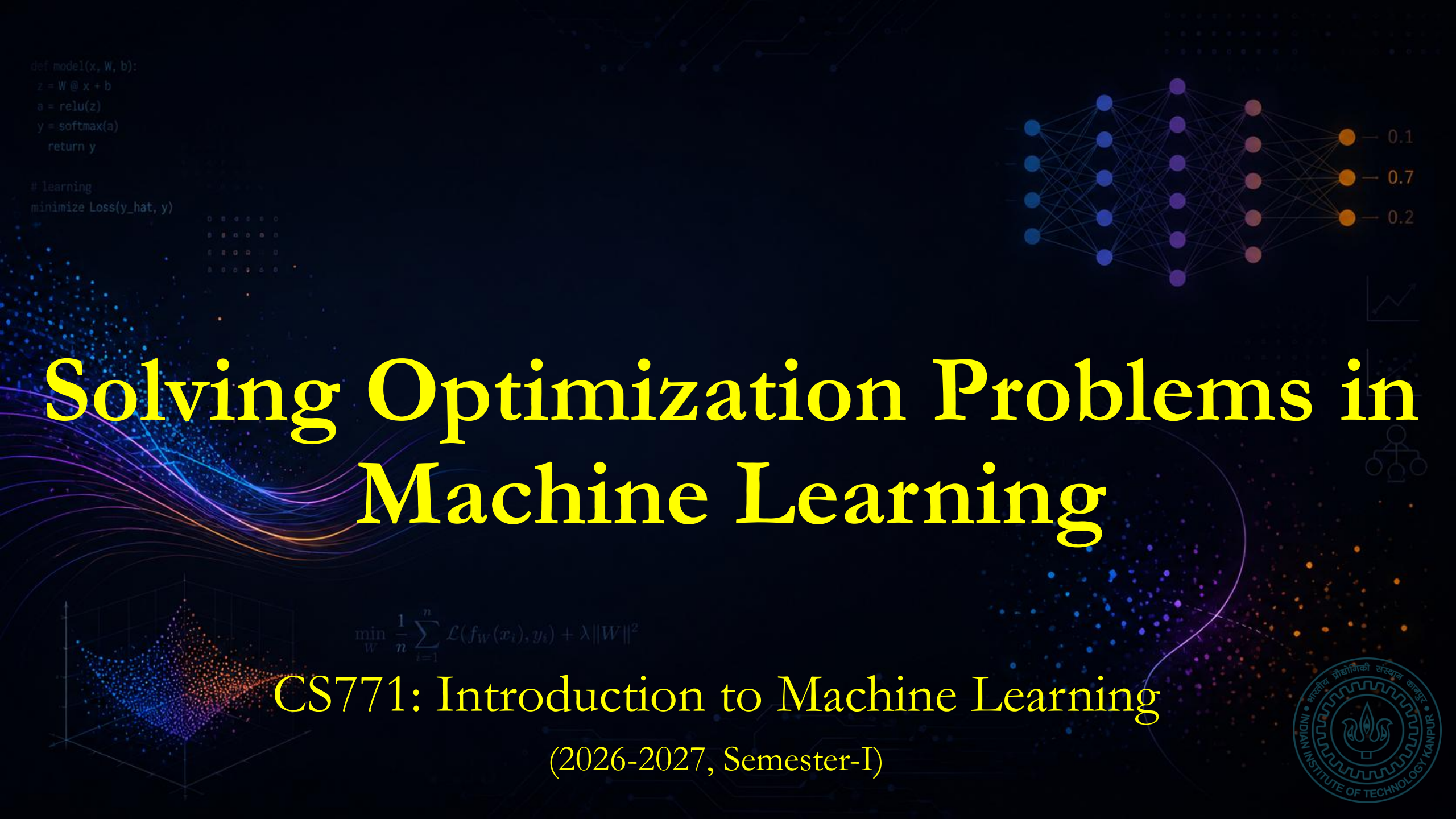


Solving Optimization Problems in Machine Learning

CS771: Introduction to Machine Learning

(2026-2027, Semester-I)

Solving Optimization Problems in ML

Last class: For linear model with weight $\boldsymbol{w}$, bias $\boldsymbol{b}$, and loss func $\mathcal{L}(\boldsymbol{w}, \boldsymbol{b})$

$$(w_{\text{opt}}, b_{\text{opt}}) = \arg\min_{w,b} \mathcal{L}(\boldsymbol{w}, \boldsymbol{b})$$

In general, for an ML model with params $\boldsymbol{w} \in \mathbb{R}^D$ and loss func $\mathcal{L}(\boldsymbol{w})$

$$w_{\text{opt}} = \arg\min_{w} \mathcal{L}(\boldsymbol{w})$$

Note: The parameters $\boldsymbol{w}$ may have constraints – will discuss that later



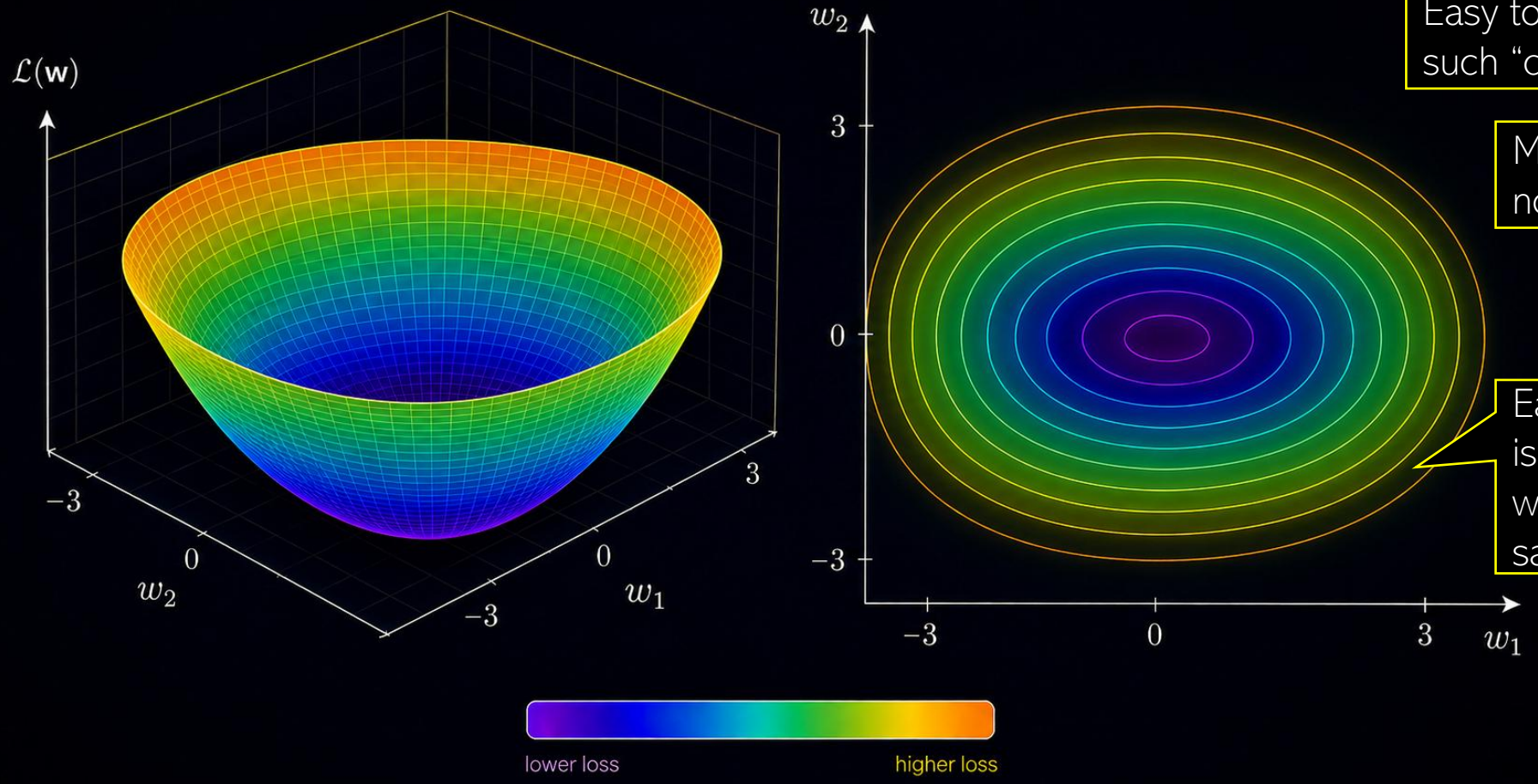
Visualization of a Simple Loss Function

Surface and contour plot of a simple loss function

Simple because of its simple bowl-like shape and presence of a single unique optima

Easy to find the optima of such "convex" functions

More on convexity and non-convexity later



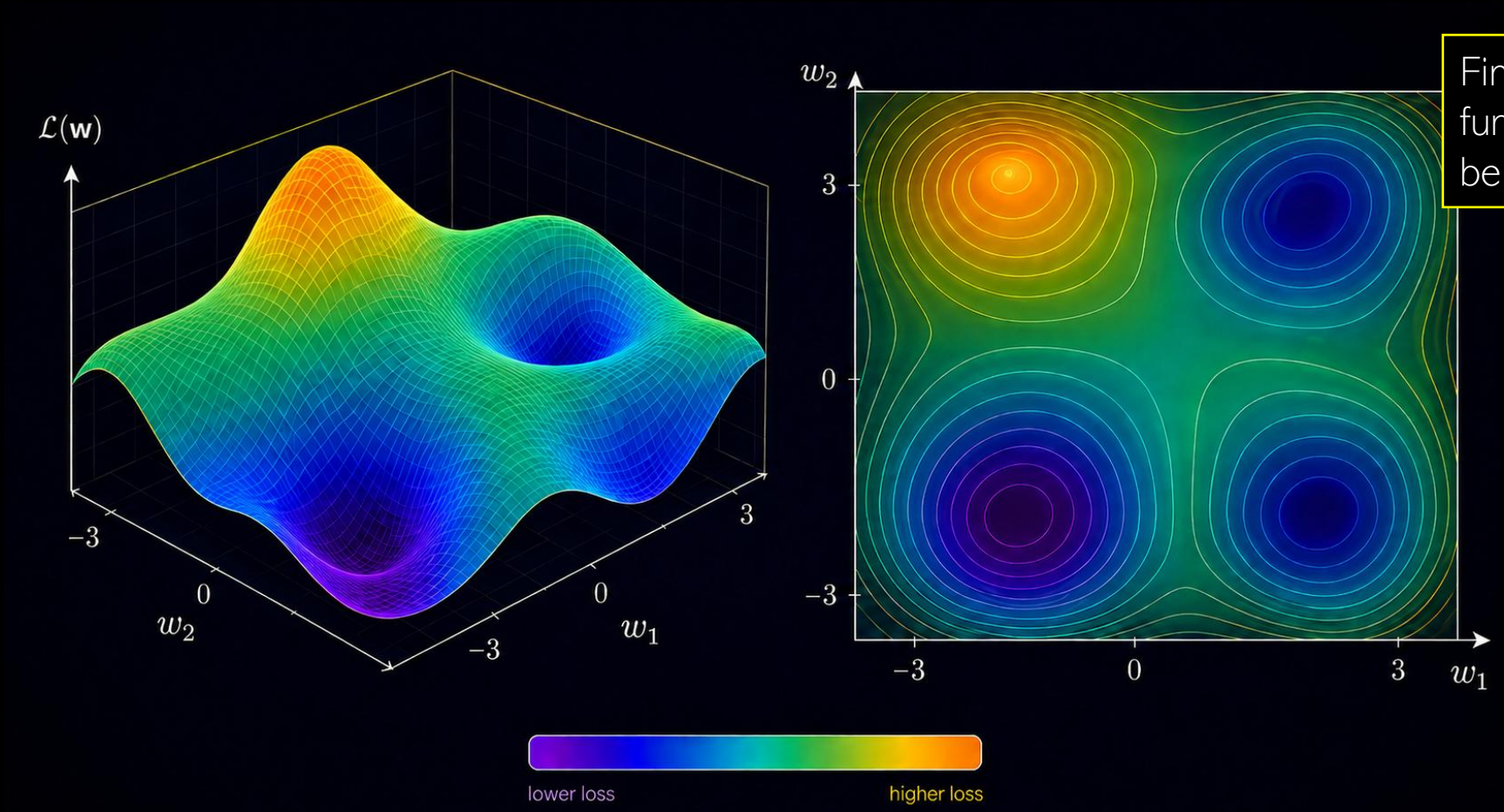
Each curve on a contour is the set of all points where the loss has the same value

Visualization of a Not-So-Simple Loss Function

Surface and contour plot of a not-so-simple loss function

Not-so-simple because of its complex shape and presence of multiple optima

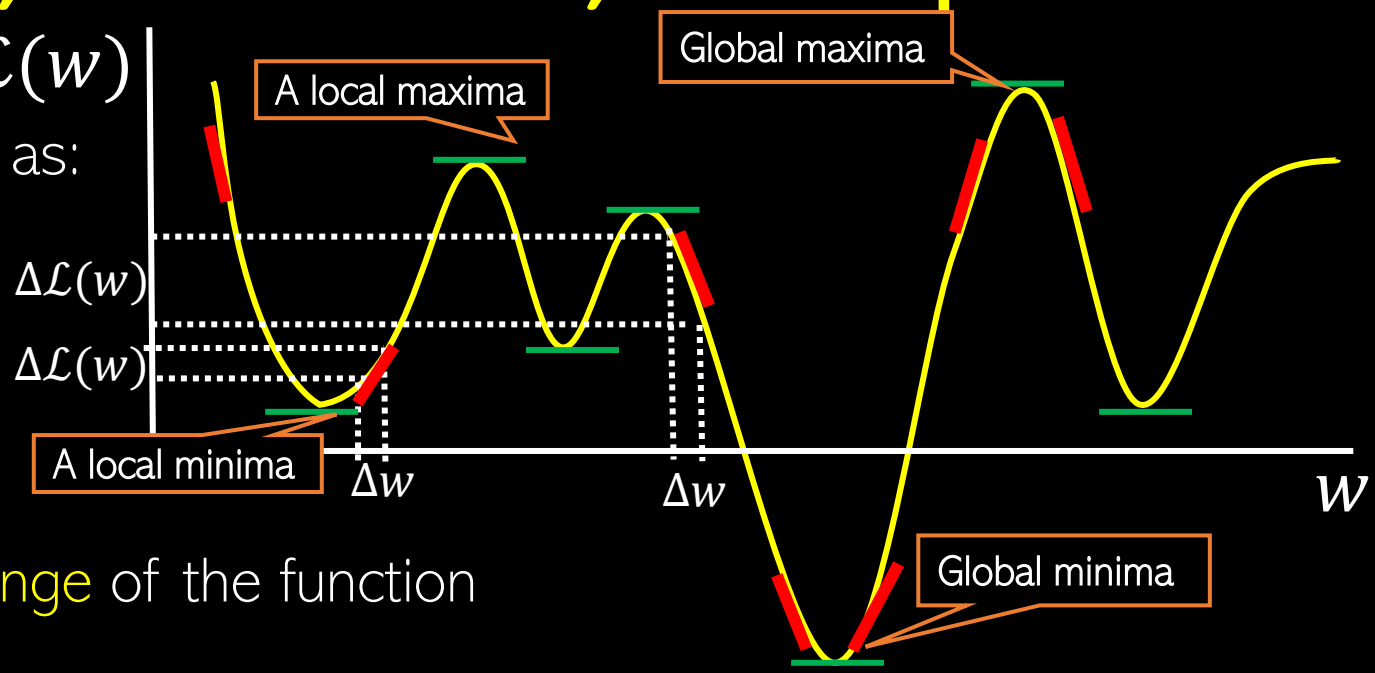
Finding the optima of such functions may require us to be more careful



Background: Functions, Derivatives, and Optima

Derivative of a function $\mathcal{L}: \mathbb{R} \rightarrow \mathbb{R}$ is defined as:

$$\mathcal{L}'(w) = \frac{d\mathcal{L}(w)}{dw} = \lim_{\Delta w \to 0} \frac{\Delta \mathcal{L}(w)}{\Delta w}$$



Derivative or "slope" denotes the rate of change of the function

Derivative/slope can be positive, negative, or zero

Function's stationary points are where the derivative is zero

Or "saddle points" – will see later

Stationary points can be function's optima (maxima or minima)

May have multiple local max/min (but only one global min/max)

Maxima at w: $\mathcal{L}'(w) > 0$ before it and < 0 after it

Minima at w: $\mathcal{L}'(w) < 0$ before it and > 0 after it

Second derivative $\mathcal{L}''(w)$ can also be used to check whether the point is a maxima or minima

Derivatives of Function with Many Variables

In ML, we often come across a scalar-valued function $\mathcal{L}\colon \mathbb{R}^D \to \mathbb{R}$ of D variables $\boldsymbol{w} = [w_1, w_2, \ldots, w_D]^\top$

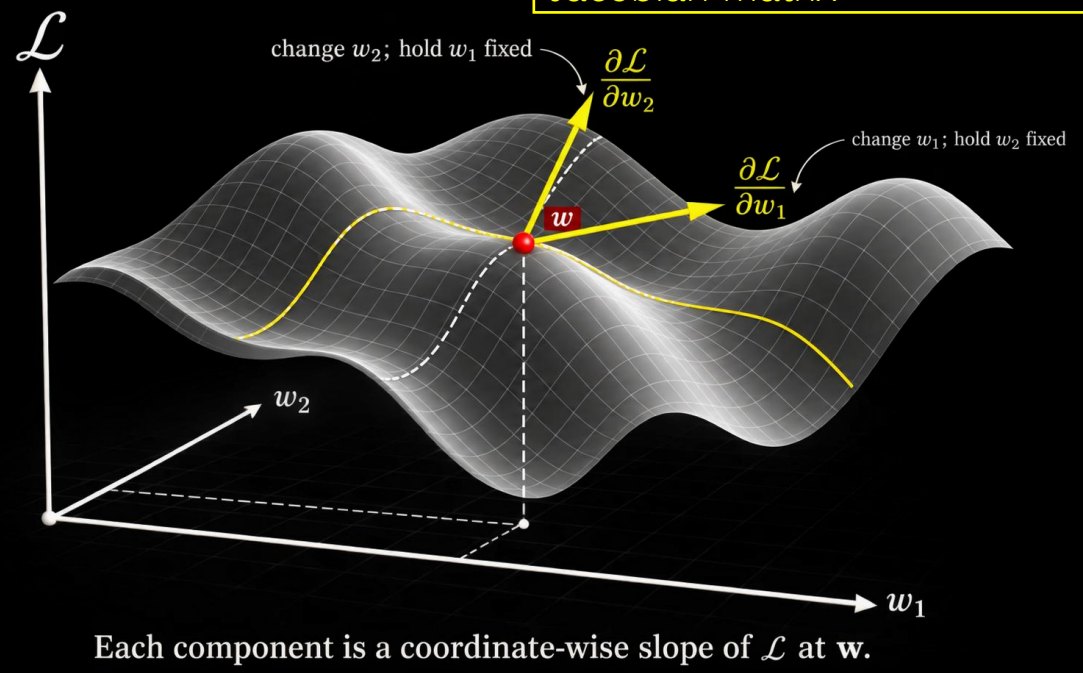
Loss functions are a common example of this

Note: If the function itself is vector-valued $\mathcal{L}\colon \mathbb{R}^D \to \mathbb{R}^M$, the analogous quantity to represent its derivatives is the Jacobian matrix

Derivative of such a function is called the gradient and is defined as

$$\boldsymbol{g} = \nabla_{\boldsymbol{w}}\mathcal{L}(w) = \begin{bmatrix} \partial\mathcal{L}/\partial w_1 \\ \partial\mathcal{L}/\partial w_2 \\ \vdots \\ \partial\mathcal{L}/\partial w_D \end{bmatrix}$$

Partial derivatives w.r.t. each of the D variables



Each component is a coordinate-wise slope of $\mathcal{L}$ at $\mathbf{w}$.

At the stationary point(s), the gradient must be zero (akin to the scalar $\boldsymbol{w}$ case)

The key property to use gradients for optimization

$$\boldsymbol{g} = \nabla_{\boldsymbol{w}}\mathcal{L}(w) = \mathbf{0}$$

Sometimes, just solving this relation can give a nice closed form solution for the optimal value of $\boldsymbol{w}$

If that's not easy, we can still use the gradients in an iterative procedure to find the optimal $\boldsymbol{w}$

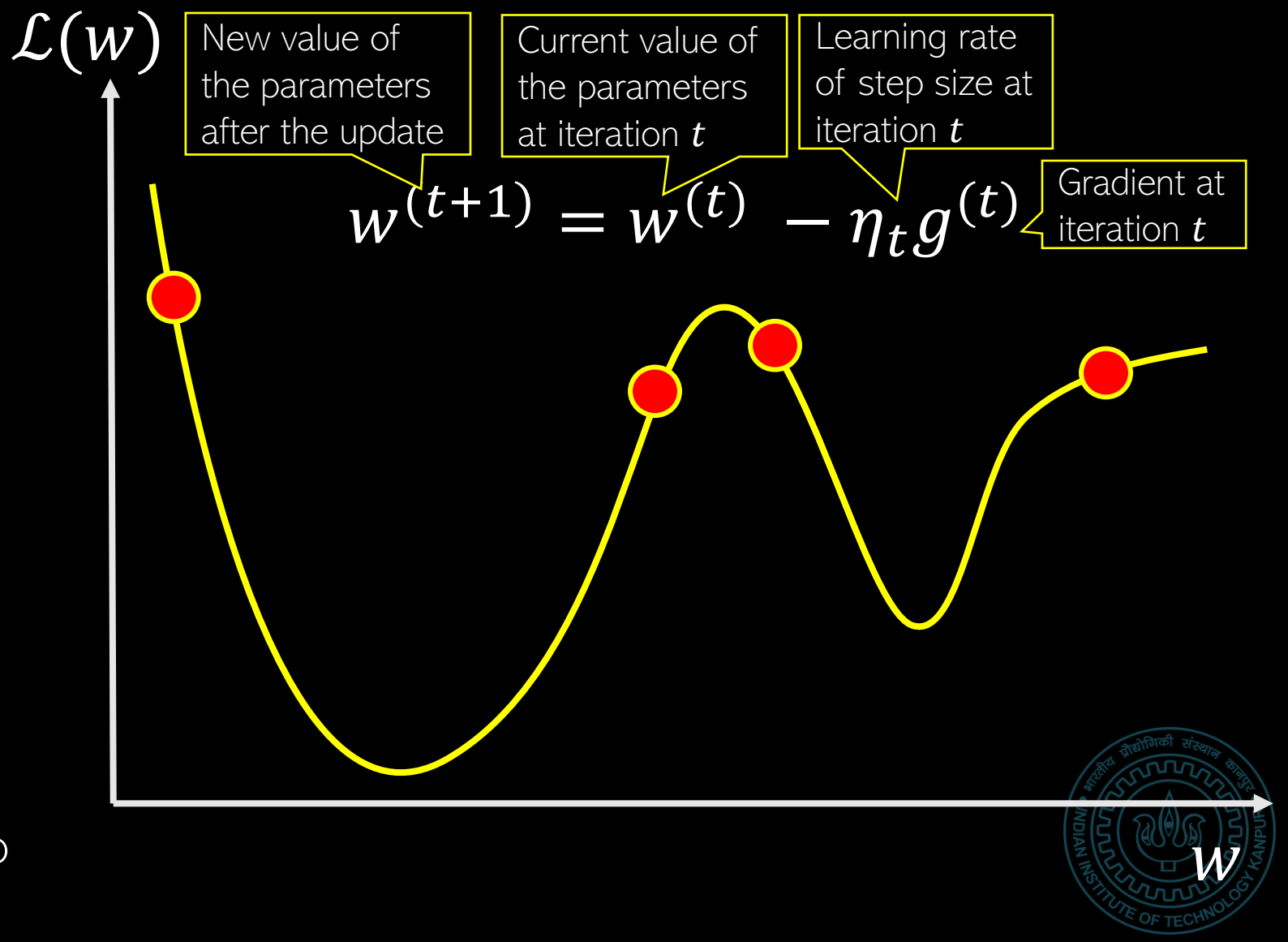
Using Gradients for Iterative Optimization

Moving in the opposite direction of the slope (derivative/gradient) moves us towards the minima

Can therefore use a simple iterative procedure using gradients to find the minima

The procedure is guaranteed to converge to a local minima

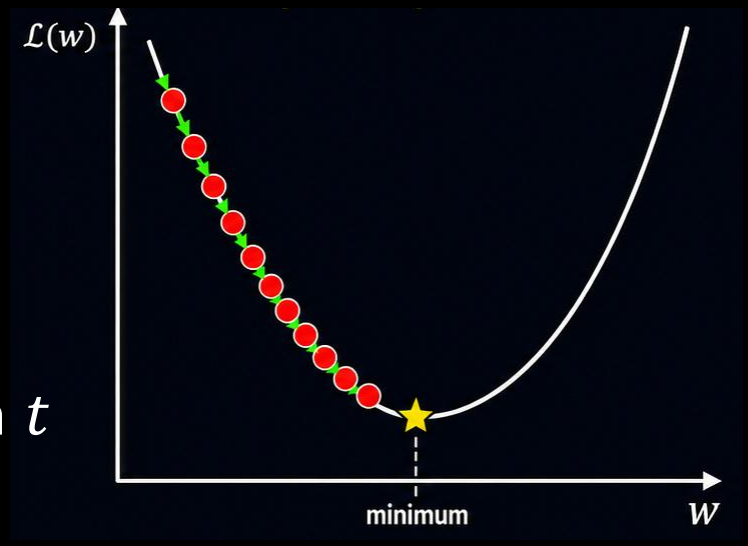
Initialization matters and controls which local minima it converges to



The Right Step-Size/Learning Rate?

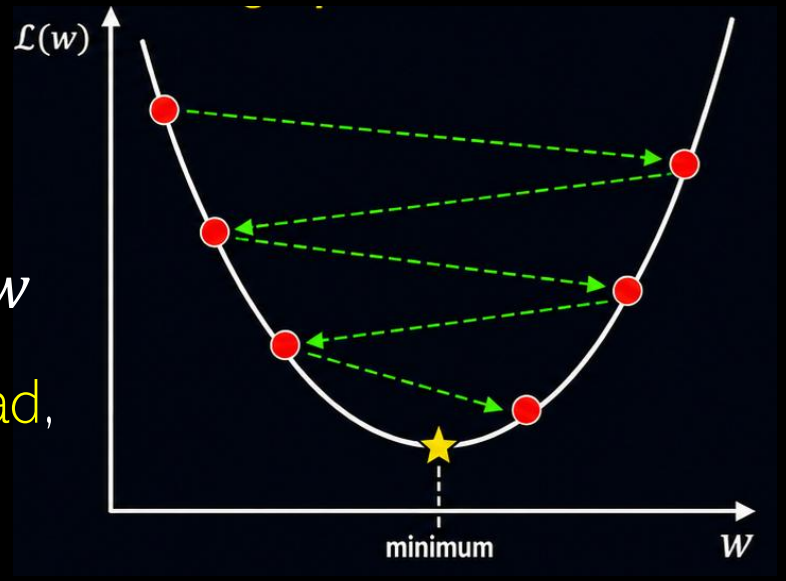
The learning rate (LR) plays a very important role

$$w^{(t+1)} = w^{(t)} - \eta_t g^{(t)}$$



Some things to keep in mind about LR

- We can use a constant LR η for all iterations or vary it with t
- Very small LR may lead to very slow convergence
- Very large LR may lead to oscillatory behavior
- The right LR can be chosen using cross-validation
- We can even have different LRs for different dimensions of w
- Advanced iterative optimization algos such as Adam, Adagrad, etc use cleverly defined LRs (will discuss them later)



Gradient Descent: The Generic Template

Remember: good initialization is important

The same template can be used if we are finding the maxima of the function and the algorithm is called gradient ascent with update equation

$$\boldsymbol{w}^{(t+1)} = \boldsymbol{w}^{(t)} + \eta_t \boldsymbol{g}^{(t)}$$

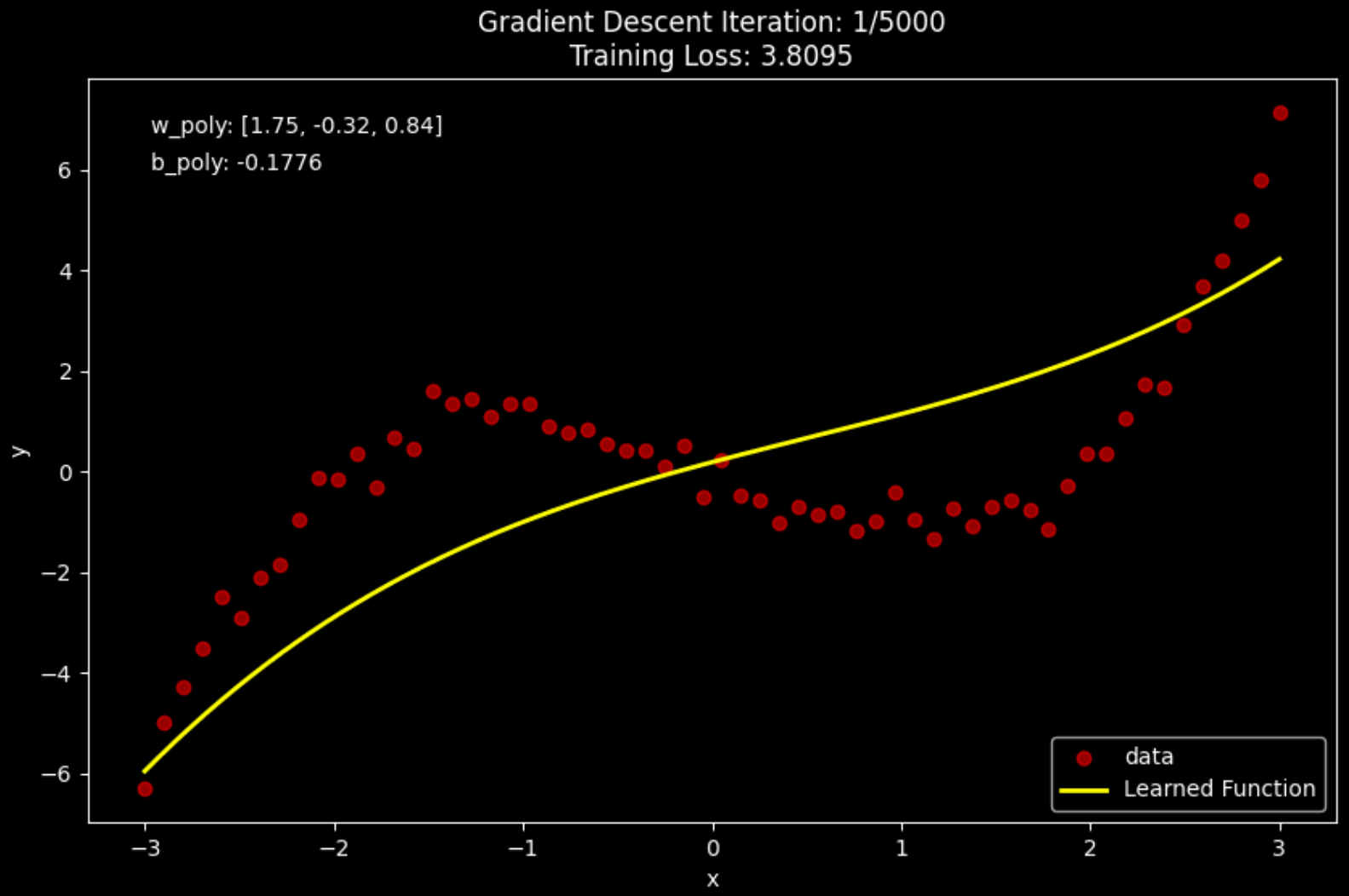


- Initialize $\boldsymbol{w}$ as $\boldsymbol{w}^{(0)}$
- For iteration $t = 0,1,2, \ldots$ (or until convergence)
  - Calculate gradient $\boldsymbol{g} = \nabla_{\boldsymbol{w}} \mathcal{L}(\boldsymbol{w})$ at the current iterate $\boldsymbol{w}^{(t)}$
  - Set the learning rate η_t
  - Move in the opposite direction of gradient

$$\boldsymbol{w}^{(t+1)} = \boldsymbol{w}^{(t)} - \eta_t \boldsymbol{g}^{(t)}$$



Gradient Descent in Action on a Toy Problem



A Closer Look at Gradient Descent Updates

- Gradient for linear regression squared loss $\mathcal{L}(\boldsymbol{w}) = \frac{1}{N}\sum_{n=1}^{N}(y_n - \boldsymbol{w}^\top \boldsymbol{x}_n)^2$

$$g = \nabla\mathcal{L}(\boldsymbol{w}) = -\frac{2}{N}\sum_{n=1}^{N}(y_n - \boldsymbol{w}^\top \boldsymbol{x}_n)^2 \boldsymbol{x}_n$$

Chain rule of calculus applied

- Iterative updates for linear regression will be of the form

$$\boldsymbol{w}^{(t+1)} = \boldsymbol{w}^{(t)} - \eta_t \boldsymbol{g}^{(t)}$$

$$= \boldsymbol{w}^{(t)} + \eta_t \frac{2}{N}\sum_{n=1}^{N}\left(y_n - \boldsymbol{w}^{(t)\top}\boldsymbol{x}_n\right)\boldsymbol{x}_n$$

Note the form of each term in the gradient expression update: Amount of current $\boldsymbol{w}$'s error on the nth training example multiplied by the input $\boldsymbol{x}_n$

This also indicates that the final $\boldsymbol{w}$ can be written as $\boldsymbol{w} = \sum_{n=1}^{N}\alpha_n \boldsymbol{x}_n$

.. because each update is adding to current $\boldsymbol{w}$ a term of the form "something times $\boldsymbol{x}_n$"

Which indicates that we are making progress towards the correct $\boldsymbol{w}$

- This update is also "corrective" in nature
  - Can show that y_n is closer to $\boldsymbol{w}^{(t+1)\top}\boldsymbol{x}_n$ than to $\boldsymbol{w}^{(t)\top}\boldsymbol{x}_n$

GD updates for other loss functions also have the same corrective nature

Gradient Descent: Two Key Assumptions

GD assumes that the function $\mathcal{L}(\boldsymbol{w})$ is "differentiable"

- Reason: We need to calculate derivatives/gradients so the function to be optimized needs to be differentiable

The parameters $\boldsymbol{w}$ being optimized for are "unconstrained"

$\boldsymbol{w}$ is scalar (i.e., real-valued) or vector with scalar entries

- Reason: Form of GD update $\boldsymbol{w}^{(t+1)} = \boldsymbol{w}^{(t)} - \eta_t \boldsymbol{g}^{(t)}$ mandates that $\boldsymbol{w}$ will be real-valued



Does it mean I can't use GD if my ML problem doesn't meet these assumption?

Can use "sub-gradient descent" if $\mathcal{L}(\boldsymbol{w})$ is non-differentiable

No, you may still use the same GD template with some simple modifications

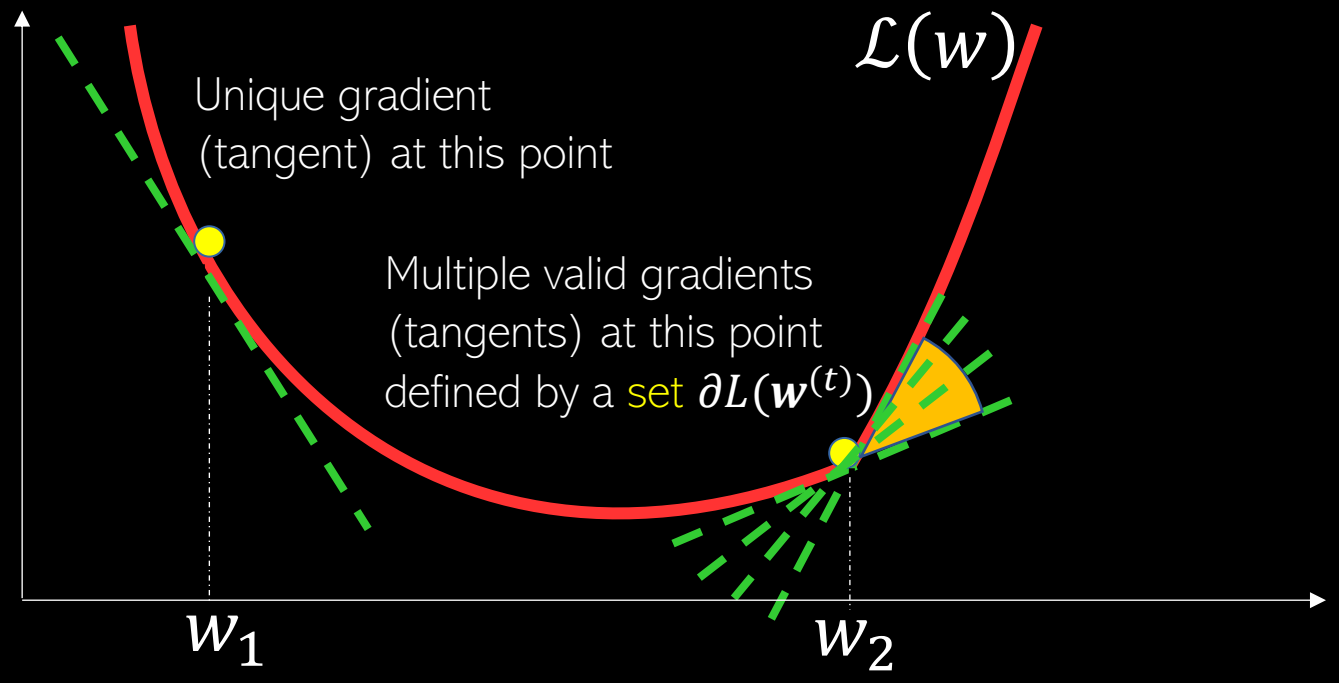
Can use projected gradient descent or use GD with $\boldsymbol{w}$ parametrized as a transformation of an unconstrained variable $\boldsymbol{z}$ if $\boldsymbol{w}$ belongs to a constraint set $\mathcal{C}$



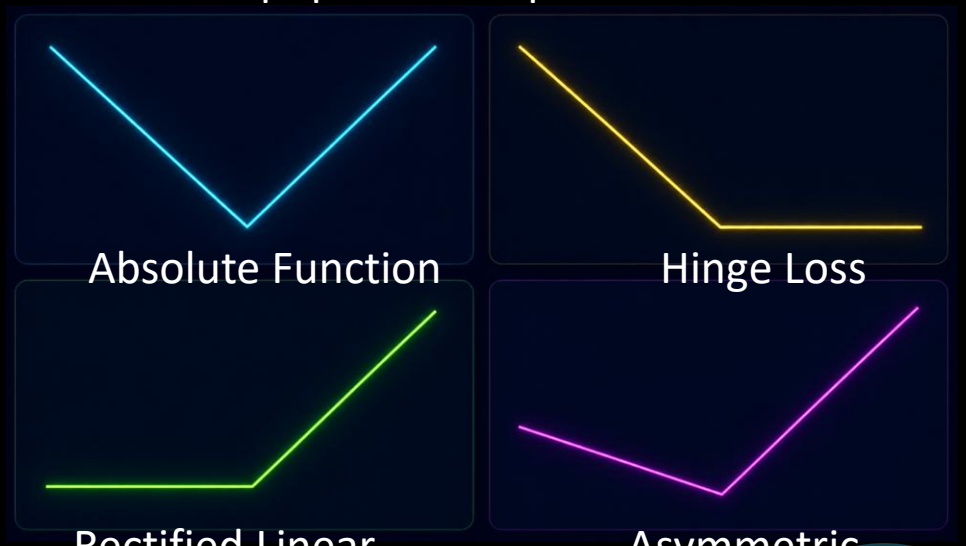
Non-Differentiable Loss Functions

Non-differentiable functions are those that have one or more non-smooth edges

We can't define a unique gradient at such points, thus such functions are called non-differentiable



Some popular examples from ML



GD for Non-Differentiable Loss Functions

Sub-gradient Descent is the common strategy

Can think of Sub-gradient Descent as a generalized version of GD

"Instead of gradient, just use any of the possible sub-gradients"

- Initialize $\boldsymbol{w}$ as $\boldsymbol{w}^{(0)}$
- For iteration $t = 0,1,2, \ldots$ (or until convergence)
  - Calculate the sub-gradient $\boldsymbol{g}^{(t)} \in \partial L(\boldsymbol{w}^{(t)})$
  - Set the learning rate η_t
  - Move in the opposite direction of subgradient

$$\boldsymbol{w}^{(t+1)} = \boldsymbol{w}^{(t)} - \eta_t \boldsymbol{g}^{(t)}$$

At other points, just pick any member from this set and use it as the gradient

Note: at points where the function is differentiable, $\partial L(\boldsymbol{w}^{(t)})$ is a singleton set anyway

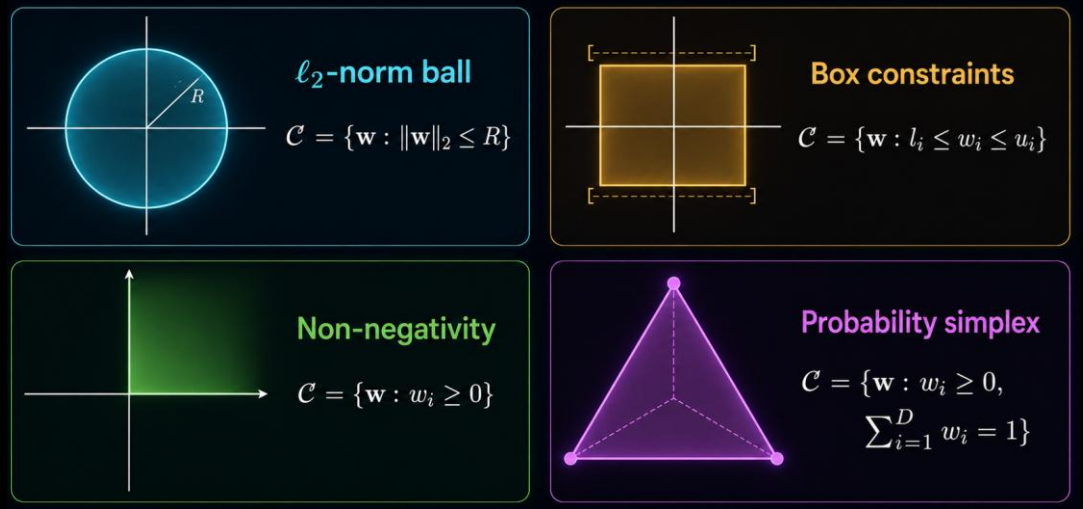


Optimization with Constraints on Variables

$$w_{opt} = \arg\min_{w \in \mathcal{C}} L(\boldsymbol{w})$$

Some popular examples of the constraint set $\mathcal{C}$

- ℓ_2 norm ball constraint: $\mathcal{C} = \{\boldsymbol{w}: \|\boldsymbol{w}\|^2 \leq R\}$
- Box Constraints: $\mathcal{C} = \{\boldsymbol{w}: l_i \leq w_i \leq u_i\}$
- Non-negativity constraints: $\mathcal{C} = \{\boldsymbol{w}: w_i \geq 0\}$
- Probability Simplex constraints: $\mathcal{C} = \{\boldsymbol{w}: \sum_{i=1}^{D} w_i = 1\}$



Some Ways to Solve Constrained Optimization

Idea 1: Reparameterization + GD

- Will discuss today - transforming constrained variables into unconstrained ones to leverage standard optimizers like GD.

Idea 2: Projected Gradient Descent (PGD)

- Will discuss today – If GD updates throw us out of the constraint set, map the updates back to constraint set by a "projection" step

Idea 3: Lagrangian-based Optimization

- Not today; will see later - moving constraints into the objective function via dual variables to solve complex coupled constraints.



Constrained Optimization

Idea 1: Reparametrize the constrained variables using a suitable transformation t

$$a \in \mathbb{R} \longrightarrow \boxed{t} \longrightarrow w = t(a) \in \mathcal{C}$$

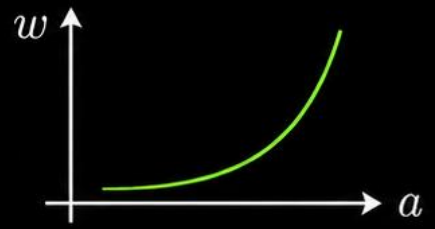
$$\text{minimize}_a \quad \tilde{\mathcal{L}}(a) = \mathcal{L}(t(a))$$

Use standard gradient descent on a

$$a^* \longrightarrow w^* = t(a^*) \in \mathcal{C}$$

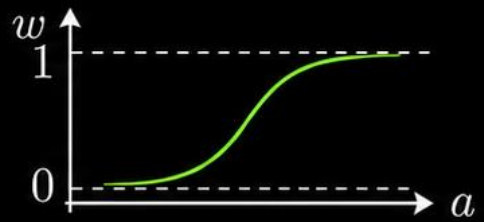
$w > 0$

$w = \exp(a)$



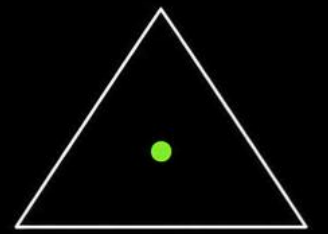
$0 < w < 1$

$w = \sigma(a)$



probability vector

$w = \text{softmax}(a)$



Constrained Optimization

PGD is efficient and very popular when the projection step $\Pi_{\mathcal{C}}[\boldsymbol{z}]$ is cheap, e.g., non-negativity or box constraints

For non-negativity constraints, $\Pi_{\mathcal{C}}[\boldsymbol{z}]$ simply means setting the negative entries of $\boldsymbol{z}$ to 0



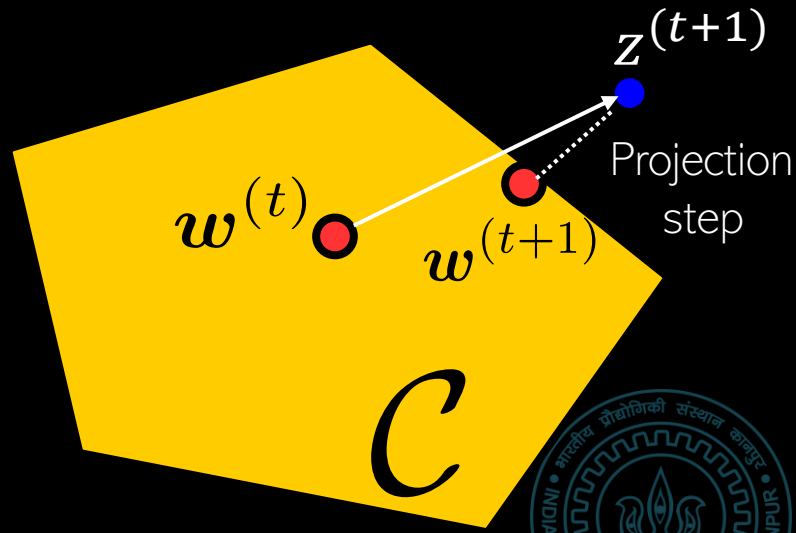
Idea 2: Use Projected Gradient Descent (PGD)

Projected GD is very similar to GD with an extra projection step

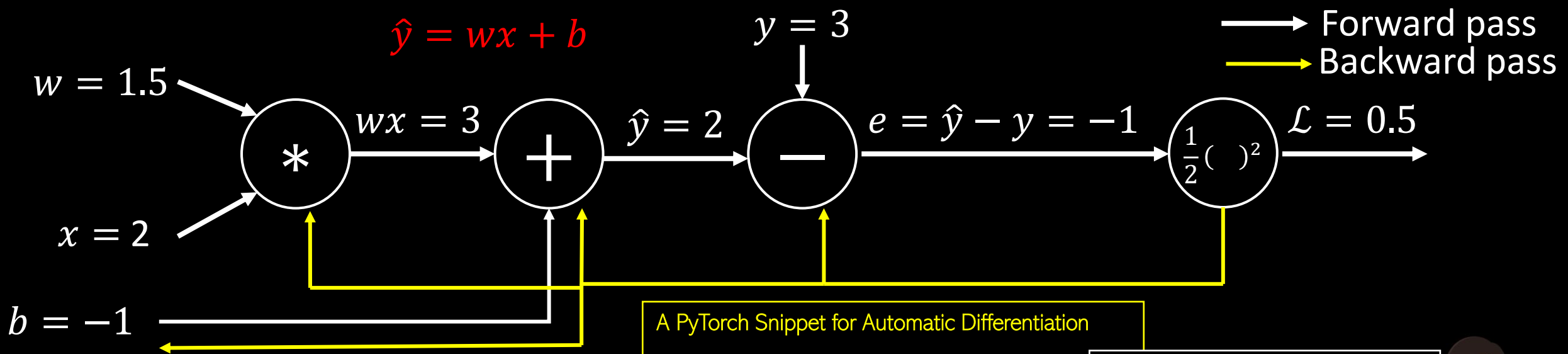
Each PGD iteration t will be of the form

- Perform update: $\boldsymbol{z}^{(t+1)} = \boldsymbol{w}^{(t)} - \eta_t \boldsymbol{g}^{(t)}$
- Check if $\boldsymbol{z}^{(t+1)}$ satisfies constraints
  - If $\boldsymbol{z}^{(t+1)} \in \mathcal{C}$, set $\boldsymbol{w}^{(t+1)} = \boldsymbol{z}^{(t+1)}$
  - If $\boldsymbol{z}^{(t+1)} \notin \mathcal{C}$, project as $\boldsymbol{w}^{(t+1)} = \Pi_{\mathcal{C}}[\boldsymbol{z}^{(t+1)}]$

$$\Pi_{\mathcal{C}}[\boldsymbol{z}] = \arg\min_{\boldsymbol{w} \in \mathcal{C}} \|\boldsymbol{z} - \boldsymbol{w}\|^2$$



Computing Gradients: Automatic Differentiation



$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial e} * \frac{\partial e}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial w} = (-1) * 1 * 2 = -2$$

$$\frac{\partial \mathcal{L}}{\partial b} = \frac{\partial \mathcal{L}}{\partial e} * \frac{\partial e}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial b} = (-1) * 1 * 1 = -1$$

A PyTorch Snippet for Automatic Differentiation

```
x = torch.tensor(2.0)
y = torch.tensor(3.0)

w = torch.tensor(1.5, requires_grad=True)
b = torch.tensor(-1.0, requires_grad=True)

y_pred = w * x + b

loss = 0.5 * (y_pred-y)**2

loss.backward()

w.grad
b.grad
```

Once gradients are computed, any suitable optimizer can make use of the gradients



We do the forward pass and loss computation

Automatic differentiation does the backward pass (computed gradients)

Gradient values

Faster GD: Stochastic Gradient Descent (SGD)

- Consider a loss function of the form $\mathcal{L}(\boldsymbol{w}) = \frac{1}{N}\sum_{n=1}^{N} \ell_n(\boldsymbol{w})$

Writing as an average instead of sum. Won't affect minimization of $L(\boldsymbol{w})$

- The gradient in this case can be written as

$$\boldsymbol{g} = \nabla_{\boldsymbol{w}} L(w) = \nabla_{\boldsymbol{w}}\left[\frac{1}{N}\sum_{n=1}^{N} \ell_n(\boldsymbol{w})\right] = \frac{1}{N}\sum_{n=1}^{N} \boldsymbol{g}_n$$

Expensive to compute – requires doing it for all the training examples in each iteration ☹

Gradient of the loss on n^{th} training example

- Stochastic Gradient Descent (SGD) approximates $\boldsymbol{g}$ using a single training example
- At iter. $\boldsymbol{t}$, pick an index $\boldsymbol{i} \in \{1,2,\ldots,N\}$ uniformly randomly and approximate $\boldsymbol{g}$ as

$$\boldsymbol{g} \approx \boldsymbol{g}_i = \nabla_{\boldsymbol{w}} \ell_i(\boldsymbol{w})$$

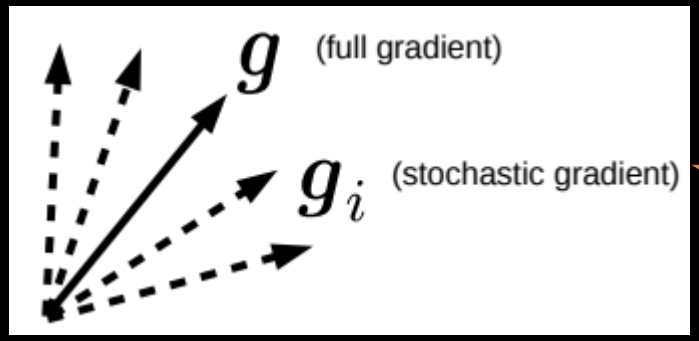
Can show that $\boldsymbol{g}_i$ is an unbiased estimate of $\boldsymbol{g}$, i.e., $\mathbb{E}[\boldsymbol{g}_i] = \boldsymbol{g}$



- May take more iterations than GD to converge but each iteration is much faster ☺
  - SGD per iter cost is $O(D)$ whereas GD per iter cost is $O(ND)$

More Robust SGD: Mini-Batch GD

- Gradient approximation using a single training example may be noisy



The approximation may have a high variance – may slow down convergence, updates may be unstable, and may even give sub-optimal solutions (e.g., local minima where GD might have given global minima)

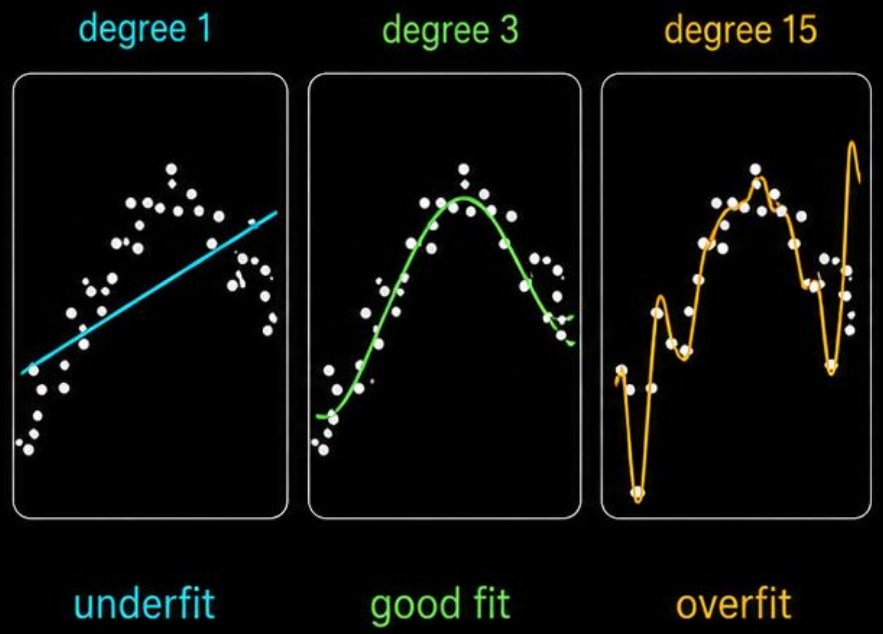
- We can use $B > 1$ unif. rand. chosen train. ex. with indices $\{i_1, i_2, \ldots, i_B\} \in \{1,2, \ldots, N\}$
- Using this "minibatch" of examples, we can compute a minibatch gradient

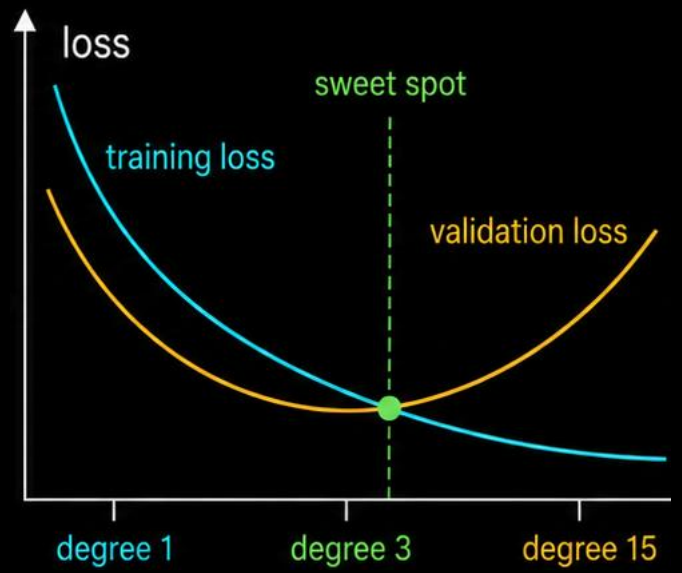
$$\boldsymbol{g} \approx \frac{1}{B} \sum_{b=1}^{B} \boldsymbol{g}_{i_b}$$

- Averaging helps in reducing the variance in the stochastic gradient
- Time complexity is $O(BD)$ per iteration in this case



Don’t Optimize Just for Training Loss







Can we train a degree 15 model and yet control its complexity?

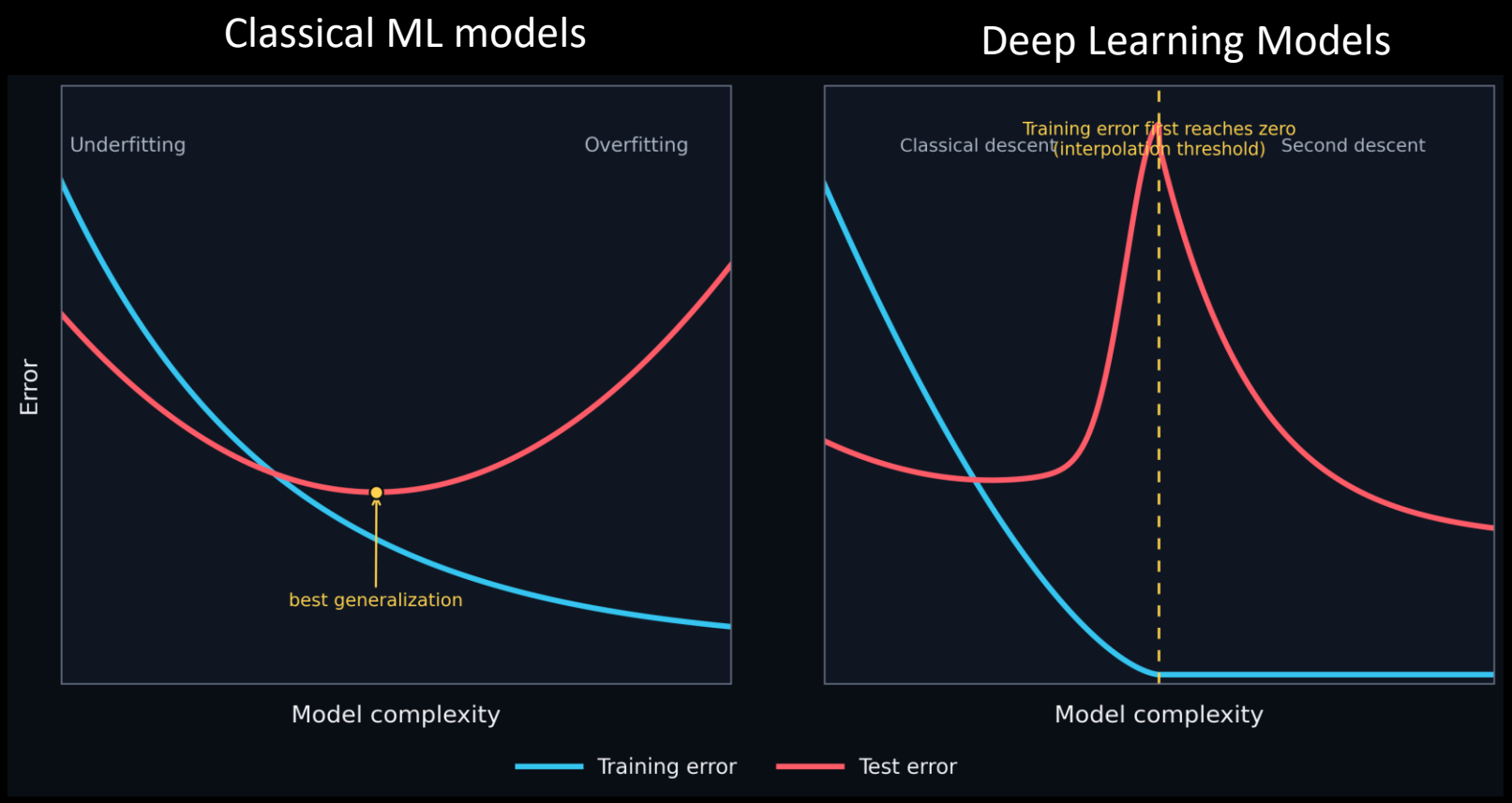
Regularization is a general method to keep model complexity low, reduce overfitting, and improve generalization

Yes, regularization enables us to do this!



But there is a caveat (for deep learning models)

Monitoring the training and validation/test errors doesn't tell the full story about overfitting in deep learning models due to "double descent" phenomena



More on this when we discuss deep learning in detail



Learning Regularized Models

Regularizing an ML model ensures that it doesn't overfit the training data and generalizes well on unseen future data. Our optimization goal will be

$$w_{\text{opt}} = \arg\min_w \mathcal{L}(\boldsymbol{w})$$

More on regularization in the next class

(such that $\boldsymbol{w}$ remains "simple" and doesn't get obsessed to get very small training loss)

Many ways to ensure "simplicity" of the params $\boldsymbol{w}$, such as

- Regularization

Hyperparameter which controls how well we want to fit the training data vs how simple we want the weights to be

$$w_{\text{opt}} = \arg\min_w \mathcal{L}(\boldsymbol{w}) + \lambda R(\boldsymbol{w})$$

- Early stopping (stop training when validation error doesn't improve any more)
- .. and several others

